

CedarCart: test a checkout change before deciding to launch

Your guide for the lab. Start with the email, then work each card in order. Everything else is optional reference.

Contents

The request

At a glance

Before class

Part A: Is recorded satisfaction different from CedarCart's goal?

Part B: How strong is the evidence that checkout B changes purchase conversion?

Part C: Do complaints and costs change the checkout launch case?

Take-home: Does your CedarCart decision survive a careful recheck?

Finish

Checklist

Optional practice

The request

Subject: **Evidence needed for the checkout launch decision**



VP of Product and Head of Operations

To: Product Analytics Specialist (you)

Hi Product Analytics team,

We need to decide whether the new checkout is ready for a wider launch. Please separate evidence that a change exists from evidence that it is worth making. Use eligible users as the denominator, check customer complaints, and show the dollar impact.

Specifically, I need answers to 4 questions:

1. Did recorded satisfaction clear our goal?

Part A

Compare the survey mean with the stated satisfaction goal. Tell me what the test can say about the recorded respondents, and whether those respondents can stand for all our users.

2. Did checkout B improve purchase conversion?

Part B

Use the randomized A and B groups to test the difference. Show the size of the observed lift and a 95% interval. Keep the test result separate from any business hurdle.

3. Does the purchase evidence justify a launch after complaints and costs?

Part C

Compare complaint rates with our stated limit. Then account for extra purchase margin, rollout cost, and excess complaint cost. Give me a provisional decision supported by all three checks.

4. Does your decision still hold after an evidence audit?

Take-home

Recheck the workbook against the supplied formula explanations and the H1, H2, and H3 software output. Keep your own decision, but revise its explanation if a denominator, test comparison, or limitation was missing.

For your own review:

- Keep your workbook and checkpoint copies locally for your own review. This is ungraded practice; nothing is submitted.

Please make it easy to trace each claim back to a cell or an output table.

Best,

VP of Product and Head of Operations

CedarCart



cedarcart_ab_test.csv



S06P_Learning_Workbook.xlsx

S06P: Should CedarCart launch checkout Variant B?

How this lab works. You are a Product Analytics Specialist using one CedarCart CSV and one saved workbook throughout. Three 10-minute class practices apply methods taught in the main session. A 15-minute, ungraded revisit completes your own written decision in the same workbook. Set up Excel and SPSS before starting a timer.

Stakeholders: VP of Product and Head of Operations

Required tools: Excel 365 and IBM SPSS Statistics

Optional, ungraded tools: Python and R

Data: cedarcart_ab_test.csv

Status: Practice only. There is no submission, rubric or deadline.

Work	Time	Open	Visible finish check
Class practice 1: satisfaction benchmark	10 minutes	IN-CLASS_Lab Part A!B7	B11 displays a two-tailed p-value of 0.333 .
Class practice 2: conversion uplift	10 minutes	IN-CLASS_Lab Part B!B7	B11 displays 2.843 and B12 displays 0.0045 .
Class practice 3: guardrails and economics	10 minutes	IN-CLASS_Lab Part C!B7	B13 displays -\$360 ; record a brief, evidence-based decision in B15.
Completion and revisit	15 minutes	Your saved IN-CLASS_Lab Part C!B15	Write two one-sentence reasons (3 minutes), three one-sentence captions (6 minutes), and a three-sentence decision (6 minutes).

First-time file readiness uses at most **3 minutes from the existing 10-minute contingency**, leaving 7 minutes for recovery. Prepare files before class where possible. Run the supplied complete syntax once; the ten-minute practices inspect its named H1/H2/H3 output. Purple support cells are provided; yellow calculation and decision cells are your work. Written reasons and three short evidence captions belong to the existing **15-minute home revisit**, alongside the final decision.

Save and resume. Before class practice, save a working copy named `S06P_FirstName_LastName_NetID.xlsx`; keep it as your active workbook. At each checkpoint, first save the completed Part to that personalized active workbook with **Ctrl+S** on Windows or **Command-S** on Mac; then make the separately named checkpoint snapshot using **Save a Copy** or **Save As**. The activity-plan names `S06_CedarCart_Checkpoint_A.xlsx`, `S06_CedarCart_Checkpoint_B.xlsx` and `S06_CedarCart_Completed.xlsx` are required checkpoint snapshots at the three checkpoints, alongside your personalized active copy. If **Save As** switches the active file to the snapshot, reopen the now-updated personalized working file before continuing. Resume after A at `IN-CLASS_Lab Part B!B7`, after B at `IN-CLASS_Lab Part C!B7`, and after C at `IN-CLASS_Lab Part C!B15` for the revisit.

Your lab at a glance

Before class

setup
untimed

Part A

10 min
Slide 27 → 28

Part B

10 min
Slide 51 → 52

Part C

10 min
Slide 74 → 78

Take-home

15 min
At home

Finish

checklist
checklist

Before class: get ready (untimed)

- 1 Unzip `S06P_Student_Package.zip` into one folder.**
Keep every file together. SPSS looks for the CSV files next to the syntax.
- 2 Open `S06P_Learning_Workbook.xlsx` and go to the **Start Here** sheet.**
Type your first name, last name and NetID (the part before @nyu.edu). Use **File > Save As** with the workbook name it shows.
- 3 Open `S06P_SPSS.sps` in SPSS and update the `CD` command.**
Replace `<REPLACE_WITH_FOLDER_CONTAINING_CSV_AND_SPS>` with the path of your unzipped lab folder (the folder, not the `.sps` file; see [Find your folder path](#)). Save As the `.sps` name shown on Start Here.
- 4 Select the whole **H0** block, then choose **Run > Selection**.**
The Output Viewer opens with **H0 CedarCart Import and Data Screen**.

✅ **Ready when:** the H0 output shows the expected case count and your workbook is saved under your own name.

⚠️ **Stuck?** Don't start Part A on the timer. Ask for help first, or see the SPSS environment guide.

Find your folder path (for the `CD` line)

SPSS needs the path of the **folder** you unzipped, the one that holds `S06P_SPSS.sps` and the CSV files. It is **not** the path to the `.sps` file. If you open the folder and see the `.sps` file and the CSV files inside, that is the right folder.

Windows

1. Unzip first: right-click `S06P_Student_Package.zip` > **Extract All** > **Extract**. Do not work inside the zip.
2. Open the extracted `S06P_Student_Package` folder, then the **Student** folder inside it. You should now see `S06P_SPSS.sps` and the CSV files. This is your lab folder.
3. Click once in the address bar at the top of File Explorer: it turns into the path. Press **Ctrl+C**.
4. Or: go up one level, hold **Shift**, right-click that folder > **Copy as path**, then delete the double quotes it adds.

Looks like

```
C:\Users\you\Downloads\S06P_Student_Package\Student
```

Mac

1. Unzip first: double-click `S06P_Student_Package.zip` in Finder.
2. Open the extracted `S06P_Student_Package` folder, then the **Student** folder inside it. You should now see `S06P_SPSS.sps` and the CSV files. This is your lab folder.
3. Go up one level and click that folder **once** to select it (the folder, not a file inside it). Press **Option+Command+C** (Copy as Pathname), or right-click it, hold **Option** and choose **Copy ... as Pathname**.

Looks like

```
/Users/you/Downloads/S06P_Student_Package/Student
```

Paste it between the single quotes on the `CD` line:

```
CD 'C:\Users\you\Downloads\S06P_Student_Package\Student'.  
CD '/Users/you/Downloads/S06P_Student_Package/Student'.
```

* *Windows*

* *Mac*

⚠ **Common mistakes:** the path ends in `/S06P_SPSS.sps` (remove the file name); the quotes or the final period are missing; the files are still inside the zip; OneDrive or iCloud has not finished downloading the folder. If H0 says a file cannot be found, check the path first. If the path itself contains an apostrophe (`'`), put the path in double quotes instead, for example `CD "/Users/you/Kim's files/S06P_Student_Package/Student"`.

How every card works

Yellow cells: you type here Lavender cells: guidance

✓ **Check your work** box on each sheet turns to "✓ Checked"

Each card opens with a short recap of the session content it uses, with the slides and Toolkit pages to reread. Type each formula starting with `=`, then press **Enter**. The cells are already formatted, so never multiply a percentage by 100.

Part A · Is recorded satisfaction different from CedarCart's goal?

10 min

[From slide 27](#) → work this card → [Back to slide 28](#)

· bring: Calculated t-statistic of 1.00 and p-value of 0.333 confirming failure to reject the 70-point benchmark.

WHAT THE SESSION COVERED · SECTION 1 HYPOTHESIS TESTING LOGIC, ERRORS, AND BENCHMARK MEAN COMPARISONS

Start at [IN-CLASS_Lab Part A!B7](#) . Read the supplied sample size, mean, sample spread, goal, and degrees of freedom. This is a **one-sample t-test**: it compares one group's average score with a fixed goal. It does not compare checkout A with checkout B.

- **The question:** The null claim H_0 says the population mean equals the goal; the two-sided alternative H_1 says it differs.
- **The calculation:** The standard error $SE = s/\sqrt{n}$ measures uncertainty in the sample mean; $t = (\bar{x} - \mu_0)/SE$ measures its distance from the goal in standard-error units.
- **The decision:** Compare the two-sided p-value with the chosen cutoff α ; a large p-value does not prove H_0 true.

A **Type I error** rejects a true null claim. A **Type II error** misses a real difference. Neither error is fixed by calling a small survey representative. After calculating B9:B11, inspect H1 before choosing B13. At home, explain your choice in B14 and caption the H1 evidence in B17. Say clearly whether the recorded respondents can speak for all 800 users.

COURSE SLIDES

Slides 11–27

What statistical hypotheses mean in business... 12 · Two-Tailed Hypothesis Formulation and Type I Error... 13–16 · The mechanism of Type I and Type II decision errors 17 · The mathematical mechanism of the one-sample... 19 · One-Sample t-Test Statistic and Confidence Interval 20–22 · CedarCart satisfaction survey worked evidence 23

CONCISE TOOLKIT

pp. 3–7

CedarCart p. 4

FULL TOOLKIT

pp. 4–24

One-Sample t-Test for Benchmark Goals p. 13 · CedarCart p. 5

CLASS WORK AND HOME REVISIT

1. Read the supplied purple summaries B7, B8, C7, D7 and B12; enter B9:B11 during the ten-minute practice.
2. Inspect supplied H1 output, choose B13, and save the required Checkpoint A snapshot.
3. Home revisit, minutes 1–3: B14, one sentence explaining B13; reuse H1 evidence checked in class.
4. Minutes 4–9: B17, one caption sentence naming H1, its statistic and p-value, and the conclusion.

THE EQUATIONS YOU ARE BUILDING

n = number of recorded satisfaction scores · $\bar{x}$ = their mean, in points · s = their sample standard deviation, in points
· μ_0 = stated goal, in points · SE = standard error, in points · t = t statistic · df = degrees of freedom · p_t = two-sided
 t -test p -value · H_0 = no difference from the goal · H_1 = difference from the goal · α = chosen false-positive cutoff

$$SE = \frac{s}{\sqrt{n}} = \frac{C7}{\sqrt{B7}} = B9t = \frac{\bar{x} - \mu_0}{SE} = \frac{B8 - D7}{B9} = B10df = n - 1 = B12, \quad p_t = 2P(T_{df} \geq |t|) = B11$$

FIRST, THE SETUP · ONCE PER SPSS SESSION

Run this **once each time you open SPSS**, before any part. On the **CD** line, replace

<REPLACE_WITH_FOLDER_CONTAINING_CSV_AND_SPS> with the path of your **unzipped lab folder** (keep the single quotes and the final period). It is the folder that holds **S06P_SPSS.sps** and the data files, **not** the **.sps** file itself: see [Find your folder path](#). Select from the first line down to the end of the H_0 block and choose **Run > Selection**. Already ran it since you opened SPSS? Skip to the block for this part.

** Lecture connection: Cross-Tool Reconciliation and SRM Audits .*
** Why: Import the shared user-level CSV before testing or comparing outcomes .*
** How: CD identifies the folder containing the CSV; GET DATA preserves string identifiers and reads numeric fields .*
** How: FREQUENCIES shows imported group totals and recorded versus missing survey scores .*
** Read the result: Check H0's group total of 800 cases and the recorded survey N; inspect variable roles in Data Editor .*
** Watch out: Do not point CD at the syntax file or silently replace blank scores with zero .*
** Run: Open the supplied syntax with File > Open > Syntax and adjust only its CD folder .*
** Run: Select and run the complete H0 block with Run > Selection .*
** Run: Use Window to open Output Viewer; inspect H0 frequencies and Data Editor Variable View .*
** S06P supplied reproducible syntax. Replace only the CD folder below; keep CSV and syntax together.*

CD ' <REPLACE_WITH_FOLDER_CONTAINING_CSV_AND_SPS> ' . * ← change only this path

** Name the output block so you can find it in the Viewer.*

TITLE 'H0 CedarCart Import and Data Screen'.

** Import numeric fields as text first so failed conversions cannot masquerade as blank source fields.*

GET DATA

/TYPE=TXT

/FILE='cedarcart_ab_test.csv'

/ENCODING='UTF8'

/DELCASE=LINE

/DELIMITERS=","

/QUALIFIER='\"'

/ARRANGEMENT=DELIMITED

/FIRSTCASE=2

/VARIABLES=

user_id A32

group A1

purchase_raw A255

complaint_raw A255

satisfaction_raw A255

satisfaction_goal_points_raw A255

margin_per_extra_purchase_usd_raw A255

rollout_cost_per_b_user_usd_raw A255

cost_per_excess_complaint_usd_raw A255

complaint_uplift_limit_pp_raw A255.

** Give the open dataset a name so later commands can point to it.*

DATASET NAME S06P_CedarCart.

** Create or overwrite a variable with a formula, row by row.*

```

COMPUTE purchase=NUMBER(purchase_raw,F30.16).
* Create or overwrite a variable with a formula, row by row.
COMPUTE complaint=NUMBER(complaint_raw,F30.16).
* Create or overwrite a variable with a formula, row by row.
COMPUTE satisfaction=NUMBER(satisfaction_raw,F30.16).
* Create or overwrite a variable with a formula, row by row.
COMPUTE satisfaction_goal_points=NUMBER(satisfaction_goal_points_raw,F30.16).
* Create or overwrite a variable with a formula, row by row.
COMPUTE margin_per_extra_purchase_usd=NUMBER(margin_per_extra_purchase_usd_raw,F30.16).
* Create or overwrite a variable with a formula, row by row.
COMPUTE rollout_cost_per_b_user_usd=NUMBER(rollout_cost_per_b_user_usd_raw,F30.16).
* Create or overwrite a variable with a formula, row by row.
COMPUTE cost_per_excess_complaint_usd=NUMBER(cost_per_excess_complaint_usd_raw,F30.16).
* Create or overwrite a variable with a formula, row by row.
COMPUTE complaint_uplift_limit_pp=NUMBER(complaint_uplift_limit_pp_raw,F30.16).
* Create or overwrite a variable with a formula, row by row.
COMPUTE bad_purchase=(RTRIM(LTRIM(purchase_raw)) <> '' AND SYSMIS(purchase)).
* Create or overwrite a variable with a formula, row by row.
COMPUTE bad_complaint=(RTRIM(LTRIM(complaint_raw)) <> '' AND SYSMIS(complaint)).
* Create or overwrite a variable with a formula, row by row.
COMPUTE bad_purchase_domain=0.
* Compute a value only for rows that meet the condition.
IF (NOT SYSMIS(purchase)) bad_purchase_domain=(purchase <> 0 AND purchase <> 1).
* Create or overwrite a variable with a formula, row by row.
COMPUTE bad_complaint_domain=0.
* Compute a value only for rows that meet the condition.
IF (NOT SYSMIS(complaint)) bad_complaint_domain=(complaint <> 0 AND complaint <> 1).
* Create or overwrite a variable with a formula, row by row.
COMPUTE bad_satisfaction=(RTRIM(LTRIM(satisfaction_raw)) <> '' AND SYSMIS(satisfaction)).
* Create or overwrite a variable with a formula, row by row.
COMPUTE bad_goal=(RTRIM(LTRIM(satisfaction_goal_points_raw)) <> '' AND SYSMIS(satisfaction_goal_points)).
* Create or overwrite a variable with a formula, row by row.
COMPUTE bad_margin=(RTRIM(LTRIM(margin_per_extra_purchase_usd_raw)) <> '' AND
SYSMIS(margin_per_extra_purchase_usd)).
* Create or overwrite a variable with a formula, row by row.
COMPUTE bad_rollout_cost=(RTRIM(LTRIM(rollout_cost_per_b_user_usd_raw)) <> '' AND
SYSMIS(rollout_cost_per_b_user_usd)).
* Create or overwrite a variable with a formula, row by row.
COMPUTE bad_complaint_cost=(RTRIM(LTRIM(cost_per_excess_complaint_usd_raw)) <> '' AND
SYSMIS(cost_per_excess_complaint_usd)).
* Create or overwrite a variable with a formula, row by row.
COMPUTE bad_limit=(RTRIM(LTRIM(complaint_uplift_limit_pp_raw)) <> '' AND SYSMIS(complaint_uplift_limit_pp)).
COUNT malformed_fields=bad_purchase bad_complaint bad_purchase_domain bad_complaint_domain bad_satisfaction
bad_goal bad_margin bad_rollout_cost bad_complaint_cost bad_limit (1).
* Collapse rows into group summaries (one row per group).
AGGREGATE
/OUTFILE=* MODE=ADDVARIABLES
/BREAK=
/malformed_any=MAX(malformed_fields).
* Set each variable's measurement level (nominal, ordinal, scale).
VARIABLE LEVEL group purchase complaint (NOMINAL).
* Set each variable's measurement level (nominal, ordinal, scale).
VARIABLE LEVEL satisfaction satisfaction_goal_points margin_per_extra_purchase_usd rollout_cost_per_b_user_usd
cost_per_excess_complaint_usd complaint_uplift_limit_pp (SCALE).
* Run the pending transformations now.
EXECUTE.
DISPLAY DICTIONARY.

```

** Frequency table: how many cases fall in each category.*

```
FREQUENCIES VARIABLES=group purchase complaint satisfaction satisfaction_goal_points malformed_fields
malformed_any.
```

** Every case must have malformed_any=0. If not, inspect the flagged raw fields, including nonbinary purchase or complaint values, and resolve the input before interpreting H1-H3.*

```
TEMPORARY.
```

** Keep only the rows that meet the condition (others are deleted).*

```
SELECT IF (malformed_fields > 0).
```

** Print the listed variables row by row.*

```
LIST VARIABLES=user_id group bad_purchase bad_complaint bad_purchase_domain bad_complaint_domain
bad_satisfaction bad_goal bad_margin bad_rollout_cost bad_complaint_cost bad_limit.
```

** Confirm satisfaction_goal_points has one recorded value, 70, with no missing values, matching H1 TESTVAL before interpreting H1.*

BEFORE THE TIMER · INSTRUCTOR DEMO

Follow along: in SPSS, run the **H1** block and leave **H1 Satisfaction Benchmark** open.

SPSS BLOCK FOR THIS PART · ALREADY IN YOUR SYNTAX FILE

You do not type this: it is already in `S06P_SPSS.sps`, the one syntax file you use all session. Once per SPSS session, run the `CD` line and the H0 block first (the setup code is above). Then press **Ctrl+F**, search `TITLE 'H1'`, select from the first comment line down to the last period, and choose **Run > Selection**. Green lines start with `*`: they are comments that explain the command below them, and SPSS ignores them. Every command ends with a period.

** Lecture connection: One-Sample t-Test for Benchmark Goals .*

** Why: Test recorded satisfaction against the fixed source-checked 70-point goal .*

** How: TESTVAL sets the fixed goal; VARIABLES selects satisfaction and excludes blank scores .*

** How: CI(.95) requests the 95% interval for the difference from the goal .*

** Read the result: Read H1 N, mean, sample SD, t, df, two-sided significance and whether the difference interval covers zero .*

** Watch out: Do not interpret the difference-from-goal interval as a between-arm interval .*

** Run: After H0, find H1 in the supplied Syntax Editor and select the complete block .*

** Run: Choose Run > Selection, not Run > All for this single block .*

** Run: Switch with Window to Output Viewer and find H1 One-Sample Statistics and Test .*

```
TITLE 'H1 Satisfaction Benchmark'.
```

** Make the next transformation apply to the next procedure only.*

```
TEMPORARY.
```

** Keep only the rows that meet the condition (others are deleted).*

```
SELECT IF (malformed_any=0).
```

** Compare means with a t test.*

```
T-TEST /TESTVAL=70 /MISSING=ANALYSIS
/VARIABLES=satisfaction /CRITERIA=CI(.95).
```

OPEN

Open `IN-CLASS_Lab Part A!B7`.

FILL IN THE YELLOW CELLS

#	Cell	Type this	In math	Check
1	B9	<code>=C7/SQRT(B7)</code>	$SE = s/\sqrt{n}$ standard error, points	2.0
2	B10	<code>=(B8-D7)/B9</code>	$t = (x - \mu_0)/SE$ unitless test statistic	1.000

#	Cell	Type this	In math	Check
3	B11	<code>=T.DIST.2T(ABS(B10),B12)</code>	$p_t = 2P(T_{df} \geq t)$ two-sided test probability	0.333
	B13	B13: pick the best statement from the dropdown.		
4	B14	B14: Your own short interpretation of the benchmark test and whether the 16 recorded respondents represent all 800 users.		
	B17	B17: caption the SPSS H1 Satisfaction Benchmark output. The H1 table names and the N, test, and interval checks you made		

✔ **Done when:** Cell B11 displays a p-value of 0.333. **Save.**

⚠ **Most common mistake:** Treating a test that does not reject the goal as proof that every CedarCart user meets the goal. The test concerns a mean, and the recorded respondents may not represent all users.

Stuck? Three quick fixes

- **B9 looks too large.** Divide C7 by `SQRT(B7)`, not by B7.
- **B10 has an unexpected sign.** Subtract the goal in D7 from the sample mean in B8.
- **B11 is one-sided.** Use `T.DIST.2T(ABS(B10),B12)`; B12 supplies the degrees of freedom.

Why does the survey's reach matter?

The t-test describes uncertainty from having a limited number of recorded scores. It cannot repair a survey process that favored some users over others. In B14, keep the test conclusion and the question of representation in separate clauses.

Part B · How strong is the evidence that checkout B changes purchase conversion?

🕒 10 min

[From slide 51](#) → work this card → [Back to slide 52](#)

· bring: Calculated z-statistic of 2.843 and 95% CI [3.14, 16.86] percentage points for checkout conversion.

WHAT THE SESSION COVERED · SECTION 2 A/B TESTING WITH PROPORTIONS AND STATISTICAL UPLIFT

Start at IN-CLASS_Lab Part B!B7. Each arm's denominator is its eligible randomized users, not its active sessions. A **two-proportion z-test** asks whether the observed purchase-rate gap could arise when the true arm rates are equal. Under that null claim, it uses one pooled purchase rate.

- **Test:** Use the pooled rate $\hat{p}_0 = (x_A + x_B)/(n_A + n_B)$ to calculate the null standard error SE_0 , then divide the observed difference Δ by SE_0 .
- **Effect interval:** Use the supplied unpooled error SE_{Δ} around Δ for the 95% interval; it answers how large the true gap might be.
- **Units:** A difference between two rates is reported in **percentage points**, not as a relative percent change.

A small p-value supports a difference; it does not show that the entire interval clears CedarCart's business hurdle. Check the two claims separately. Enter B9:B12 and B14:B15, then inspect H2 and choose B18 without copying an answer from this guide. At home, explain the test and hurdle comparison in B19. In B22, name the H2 tables and compare its **uncorrected Pearson** result with the workbook test.

COURSE SLIDES

Slides 28–51

Randomized Units and Conversion Proportions 29 · Pooled Two-Proportion z-Test Statistic 31–33 · Excel Two-Proportion z-Test and SPSS Verification 34–35 · the Pooled Null Standard Error 36 · Calculating the z-Statistic and p-Value 37 · Mathematical Mechanism of the Unpooled Standard... 39

CONCISE TOOLKIT

pp. 8–15

CedarCart p. 9

FULL TOOLKIT

pp. 25–44

Two-Proportion z-Test Mechanics p. 25 · Unpooled Confidence Interval for Difference p. 38 · CedarCart p. 26

CLASS WORK AND HOME REVISIT

1. Read supplied counts, rates, effect and unpooled SE in purple cells; enter B9:B12 and B14:B15 during ten minutes.
2. Inspect supplied H2 output, choose B18, check B11 and B12, and save the required Checkpoint B snapshot.
3. Home revisit, minutes 1–3: B19, one sentence explaining B18; reuse H2 evidence checked in class.
4. Minutes 4–9: B22, one caption sentence naming H2, its uncorrected Pearson statistic, p-value, and conclusion.

THE EQUATIONS YOU ARE BUILDING

n_A, n_B = eligible users in arms A and B · x_A, x_B = users who purchased in each arm · p_A, p_B = observed purchase rates · $\hat{p}_0$ = pooled rate under equal true rates · SE_0 = pooled null standard error · $\Delta = p_B - p_A$ = observed rate gap · z = conversion test statistic · Φ = standard normal cumulative probability · p_z = two-sided test p-value · SE_Δ = supplied unpooled standard error · L, U = lower and upper 95% effect bounds

$$\hat{p}_0 = \frac{x_A + x_B}{n_A + n_B} = \frac{C7+D7}{B7+B8} = B9, \quad SE_0 = \sqrt{\hat{p}_0 (1 - \hat{p}_0) \left(\frac{1}{n_A} + \frac{1}{n_B} \right)} = B10$$

$$\Delta = p_B - p_A = B16, \quad z = \frac{\Delta}{SE_0} = \frac{F7-E7}{B10} = B11, \quad p_z = 2(1 - \Phi(|z|)) = B12$$

$$L = \Delta - 1.96SE_\Delta = B14, \quad U = \Delta + 1.96SE_\Delta = B15, \quad SE_\Delta = B13$$

Just opened SPSS? Run the setup first (the **CD** line and H0; code in Part A).

BEFORE THE TIMER · INSTRUCTOR DEMO

Follow along: in SPSS, run the **H2** block and leave **H2 Purchase by Group** open.

SPSS BLOCK FOR THIS PART · ALREADY IN YOUR SYNTAX FILE

You do not type this: it is already in **S06P_SPSS.sps**, the one syntax file you use all session. Once per SPSS session, run the **CD** line and the H0 block first (setup code in Part A). Then press **Ctrl+F**, search **TITLE 'H2'**, select from the first comment line down to the last period, and choose **Run > Selection**. Green lines start with *****: they are comments that explain the command below them, and SPSS ignores them. Every command ends with a period.

```
* Lecture connection: Two-Proportion z-Test Mechanics .
* Why: Compare purchase outcomes among users with observed 0 or 1 outcomes .
* How: TEMPORARY and SELECT IF retain A/B assignments with observed binary purchases for the following CROSSTABS only .
* How: CHISQ supplies the uncorrected Pearson 2x2 statistic and df 1 .
* Read the result: Read H2 counts, row percentages and the Pearson row; its uncorrected statistic matches conversion z squared .
* Watch out: Do not separate TEMPORARY and SELECT IF from CROSSTABS or compare Yates' correction with z squared .
* Run: Run H0 first; find H2 and select its entire block, including TEMPORARY and CROSSTABS .
* Run: Choose Run > Selection .
* Run: In Output Viewer, locate H2 Crosstabulation and Pearson Chi-Square, not Continuity Correction .
TITLE 'H2 Purchase by Group'.
* Make the next transformation apply to the next procedure only.
TEMPORARY.
* Keep only the rows that meet the condition (others are deleted).
SELECT IF (malformed_any=0 AND (group='A' OR group='B') AND (purchase=0 OR purchase=1)).
* Two-way table: counts and percentages for one variable by another.
CROSSTABS /TABLES=group BY purchase
/FORMAT=AVALUE TABLES /STATISTICS=CHISQ
/CELLS=COUNT ROW /COUNT ROUND CELL.
```

OPEN

Open **IN-CLASS_Lab Part B!B7**.

FILL IN THE YELLOW CELLS

#	Cell	Type this	In math	Check
1	B9	<code>=(C7+D7)/(B7+B8)</code>	$\hat{p}_0 = (x_A + x_B)/(n_A + n_B)$ pooled purchase rate	45.0%
2	B10	<code>=SQRT(B9*(1-B9)*(1/B7+1/B8))</code>	$SE_0 = \sqrt{\hat{p}_0(1-\hat{p}_0)(1/n_A + 1/n_B)}$ null standard error	0.03518
3	B11	<code>=(F7-E7)/B10</code>	$z = \Delta/SE_0$ unitless conversion statistic	2.843
4	B12	<code>=2*(1-NORM.S.DIST(ABS(B11),TRUE))</code>	$p_z = 2(1 - \Phi(z))$ two-sided test probability	0.0045
5	B14	<code>=B16-1.96*B13</code>	$L = \Delta - 1.96SE_{\Delta}$ lower effect bound	3.14%
6	B15	<code>=B16+1.96*B13</code>	$U = \Delta + 1.96SE_{\Delta}$ upper effect bound	16.86%
7	B18 B19 B22	B18: pick the best statement from the dropdown. B19: The test evidence, interval-to-hurdle comparison, and distinction between the two B22: caption the SPSS H2 Purchase by Group output. The H2 table names, observed row denominators, and uncorrected Pearson comparison you checked		

✅ **Done when:** Cell B11 displays $z = 2.843$ and B12 displays $p = 0.0045$. **Save.**

⚠️ **Most common mistake:** Using the pooled standard error for the effect interval. B10 is for testing equal rates; the interval uses the supplied unpooled standard error in B13.

Stuck? Three quick fixes

- **The pooled rate exceeds 100%.** Divide total purchases by total eligible users in B9; do not multiply the formula by 100.
- **The z sign seems reversed.** B11 uses the B rate in F7 minus the A rate in E7.
- **H2 does not match B11.** Compare B11 squared with H2's uncorrected Pearson chi-square, not its continuity-corrected row.

Why are the two standard errors different?

The test temporarily assumes equal true purchase rates, so it combines the purchases to estimate one rate. The effect interval does not make that equality assumption. It estimates uncertainty using each arm's observed rate. That is why B10 and B13 have different jobs.

Part C · Do complaints and costs change the checkout launch case?

🕒 10 min

[From slide 74](#) → work this card → [Back to slide 78](#)

· bring: Calculated net loss of -\$360 and verified guardrail breach leading to recommendation against rollout.

WHAT THE SESSION COVERED · SECTION 3 PRACTICAL SIGNIFICANCE, GUARDRAIL METRICS, AND CROSS-TOOL RECONCILIATION

Start at IN-CLASS_Lab Part C!B7 . A **guardrail** is a limit on harm that matters alongside the main purchase measure. First calculate complaints per eligible user in each arm. Then compare the B-minus-A complaint gap with the supplied limit in B12. Finally calculate the experiment's net dollar impact.

- **Complaint rates:** Each arm's complaint count is divided by that arm's eligible users; the gap Δ_c is in percentage points.
- **Net impact:** Value extra purchases at margin, then subtract B rollout cost and the cost of excess complaints.
- **Reconcile:** For a two-by-two table, the uncorrected Pearson result χ^2 matches the squared corresponding proportion statistic z^2 .

Do not let one purchase p-value make the whole launch decision. The business rule needs purchase evidence, the complaint limit, and economics together. Enter B9:B11 and B13. Inspect H3, then write **your own** provisional sentence in B15 and save the Completed snapshot. At home, add the H3 caption in B19. Expand B15 to cover the decision, both outcomes and net impact, and one limit on what this experiment establishes.

COURSE SLIDES

Slides 53–74

What practical significance and guardrails mean 54 · Net Incremental Economic Impact of Screen Variant B 55–57 · The mechanism linking practical thresholds and... 58 · CedarCart experimentation evidence and unit... 59 · Worked Cross-Tool Reconciliation on CedarCart Data 62 · SPSS Syntax for Two-by-Two Cross-Tabulation and... 64–67

CONCISE TOOLKIT

pp. 16–22

CedarCart p. 18

FULL TOOLKIT

pp. 45–68

CedarCart p. 46

CLASS WORK AND HOME REVISIT

1. Read supplied complaint counts, source limit and H3 comparison; enter B9:B11 and B13 during the ten-minute practice.
2. Inspect supplied H3 output and draft one provisional evidence-based sentence in B15; save the required Completed snapshot.
3. Home revisit, minutes 4–9: B19, one caption sentence naming H3, its uncorrected Pearson statistic, p-value, and conclusion.
4. Minutes 10–15: expand B15 to three sentences: launch decision, purchase-and-complaint evidence with net impact, and one limitation.

THE EQUATIONS YOU ARE BUILDING

n_A, n_B = eligible users in arms A and B · c_A, c_B = their complaint counts · r_A, r_B = their complaint rates · $\Delta_c = r_B - r_A$ = B-minus-A complaint-rate gap · g = supplied complaint-gap limit · $\Delta_x = B$ purchases minus A purchases · m = margin per extra purchase · c_r = rollout cost per B user · c_c = cost per excess complaint · V = observed experiment net impact · χ^2 = uncorrected Pearson statistic · z = corresponding proportion statistic

$$r_A = \frac{c_A}{n_A} = \frac{C7}{B7} = B9, \quad r_B = \frac{c_B}{n_B} = \frac{D7}{B8} = B10, \quad \Delta_c = r_B - r_A = B11, \quad g = B12$$

$$V = \Delta_x m - n_B c_r - (c_B - c_A) c_c = B13, \quad \Delta_x = \text{Part B D7} - \text{Part B C7}$$

$$\chi^2 = z^2 \quad \text{for the matching uncorrected two-by-two test}$$

Just opened SPSS? Run the setup first (the **CD** line and H0; code in Part A).

BEFORE THE TIMER · INSTRUCTOR DEMO

Follow along: in SPSS, run the **H3** block and leave **H3 Complaint by Group** open.

SPSS BLOCK FOR THIS PART · ALREADY IN YOUR SYNTAX FILE

You do not type this: it is already in **S06P_SPSS.sps**, the one syntax file you use all session. Once per SPSS session, run the **CD** line and the H0 block first (setup code in Part A). Then press **Ctrl+F**, search **TITLE 'H3'**, select from the first comment line down to the last period, and choose **Run > Selection**. Green lines start with *****: they are comments that explain the command below them, and SPSS ignores them. Every command ends with a period.

```
* Lecture connection: Cross-Tool Reconciliation and SRM Audits .
* Why: Inspect the complaint guardrail using its own observed-outcome table .
* How: TEMPORARY and SELECT IF retain A/B users with observed binary complaint outcomes .
* How: CROSSTABS reports complaint row counts and its own uncorrected Pearson statistic .
* Read the result: Compare H3's Pearson row, df 1, with Part C B14; its table is distinct from H2's purchase table .
* Watch out: Do not copy H2's chi-square into a caption about complaints .
* Run: Run H0 first; select H3 from TITLE through the final CROSSTABS period .
* Run: Choose Run > Selection .
* Run: Use Window to find H3 complaint Crosstabulation and its uncorrected Pearson row .
TITLE 'H3 Complaint by Group'.
* Make the next transformation apply to the next procedure only.
TEMPORARY.
* Keep only the rows that meet the condition (others are deleted).
SELECT IF (malformed_any=0 AND (group='A' OR group='B') AND (complaint=0 OR complaint=1)).
* Two-way table: counts and percentages for one variable by another.
CROSSTABS /TABLES=group BY complaint
/FORMAT=AVALUE TABLES /STATISTICS=CHISQ
/CELLS=COUNT ROW /COUNT ROUND CELL.
```

OPEN

Open **IN-CLASS_Lab Part C!B7**.

FILL IN THE YELLOW CELLS

#	Cell	Type this	In math	Check
1	B9	=C7/B7	$r_A = c_A/n_A$ A complaint rate	2.0%
2	B10	=D7/B8	$r_B = c_B/n_B$ B complaint rate	8.0%

#	Cell	Type this	In math	Check
3	B11	=B10-B9	$\Delta_c = r_B - r_A$ percentage-point complaint gap	6.0%
4	B13	=('IN-CLASS_Lab Part B'!D7-'IN-CLASS_Lab Part B'!C7)*INDEX(Checkout[margin_per_extra_purchase_usd],1)-COUNTIF(Checkout[group],"B")*INDEX(Checkout[rollout_cost_per_b_user_usd],1)-(D7-C7)*INDEX(Checkout[cost_per_excess_complaint_usd],1)	$V = \Delta_x m - n_B c_r - (c_B - c_A)c_c$ experiment dollars, net	-\$360

✓ **Done when:** Cell B13 displays -\$360 and B15 contains a provisional evidence-based decision sentence. Save.

⚠ **Most common mistake:** Making the launch choice from the purchase p-value alone. A statistically detectable purchase difference does not waive the complaint limit or pay the rollout and complaint costs.

Stuck? Three quick fixes

- **Complaint rates look too high.** B9 uses C7/B7 and B10 uses D7/B8 ; format the fractions as percentages without multiplying by 100.
- **The complaint gap has the wrong direction.** B11 is B10-B9 , so it is B minus A.
- **Net impact omits a cost.** In B13, include B rollout cost for every B user and excess complaint cost, as well as extra purchase margin.

Why check complaints even when purchases improve?

A checkout can make purchase easier while also creating service problems. A guardrail makes that trade-off visible before a wider launch. H3 tests whether complaint rates differ; B11 and B12 address the size of the gap against CedarCart's limit. Neither check replaces the dollar calculation in B13.

Take-home · Does your CedarCart decision survive a careful recheck?

🕒 15 min

[At home](#) → work this card → [Save locally](#)

WHAT THE SESSION COVERED · SECTION 3 PRACTICAL SIGNIFICANCE, GUARDRAIL METRICS, AND CROSS-TOOL RECONCILIATION

Start at [IN-CLASS_Lab Part C!B15](#) , where you left your provisional decision. This revisit is an **audit**, not a new experiment. Check the calculations and the words used to explain them. A p-value addresses a test claim. An interval addresses plausible effect sizes. A guardrail and net impact address the business decision.

- **Part A:** Compare B10:B11 with H1, then check that B14 does not turn evidence from recorded respondents into a claim about every user.
- **Part B:** Compare the workbook test with H2's uncorrected Pearson row; check the eligible-user denominators and explain the interval-to-hurdle comparison in B19.
- **Part C:** Compare B9:B11 and B13 with the supplied inputs, caption H3 in B19, and revise B15 in your own words.

The classic error is to treat a software label, a small p-value, or an early choice as the whole answer. Use the visible diagnostic checks to catch formula problems. Then write three short sentences in B15: your decision, the purchase and complaint evidence with net impact, and a limitation. Do not change a sound decision merely because you are revisiting it. Do change an explanation that does not match its evidence.

COURSE SLIDES

Slides 53-74

What practical significance and guardrails mean 54 · Net Incremental Economic Impact of Screen Variant B 55-57 · The mechanism linking practical thresholds and... 58 · CedarCart experimentation evidence and unit... 59 · Worked Cross-Tool Reconciliation on CedarCart Data 62 · SPSS Syntax for Two-by-Two Cross-Tabulation and... 64-67

CONCISE TOOLKIT

pp. 16-22

CedarCart p. 18

FULL TOOLKIT

pp. 45-68

CedarCart p. 46

THE EQUATIONS YOU ARE BUILDING

p_t = Part A's two-sided benchmark p-value · z = Part B's conversion statistic · χ^2_{purchase} = matching uncorrected Pearson purchase statistic · Δ_c = Part C's B-minus-A complaint-rate gap · g = Part C's supplied complaint limit · V = Part C's observed net dollar impact

$$p_t = \text{Part A B11}, \quad \chi^2_{\text{purchase}} = z^2 = (\text{Part B B11})^2 \Delta_c = \text{Part C B11}, \quad g = \text{Part C B12}, \quad V = \text{Part C B13}$$

OPEN

Open [IN-CLASS_Lab Part C!B15](#) .

FILL IN THE YELLOW CELLS

#	Cell	Build it	In math
1	B15	B15: Your decision, statistical evidence, guardrail comparison, economics, and one limitation	
	B19	B19: The H3 table names, observed complaint denominators, and uncorrected Pearson comparison you checked	

✅ **Done when:** Review IN-CLASS_Lab Part A, IN-CLASS_Lab Part B and IN-CLASS_Lab Part C against the supplied formula explanations and visible diagnostic checks; retain your own written decision. **Save.**

⚠️ **Most common mistake:** Replacing your written decision with a dropdown choice or a test result alone. B15 needs a reasoned decision that brings the purchase result, complaint guardrail, net impact, and a limitation together.

Stuck? Three quick fixes

- **H2 and Excel appear to disagree.** Square Part B B11 and use H2's uncorrected Pearson row for the comparison.
- **Your memo calls a rate gap a percent change.** Describe Part B's effect bounds and Part C B11 in percentage points.
- **B15 has only a verdict.** Add the purchase evidence, the B11-to-B12 complaint check, B13 in dollars, and one limitation.

What does cross-tool agreement establish?

Agreement between the workbook and software is a useful check on arithmetic, denominators, and the selected test row. It does not prove that a survey represents all users or that an experiment will have the same effects after a wider rollout.

Finish

Ungraded practice

Before you close the workbook

TICK EVERY BOX 0 of 17 done

Save your work

- ☐ Workbook saved under your own name (Start Here shows it), e.g. S06P_FirstName_LastName_NetID.xlsx
- ☐ Nothing to submit: this lab is ungraded practice

Part B · IN-CLASS_Lab Part B

- ☐ Cell B11 displays $z = 2.843$ and B12 displays $p = 0.0045$.
- ☐ B18 statement chosen
- ☐ B19 written: Reason for your class choice
- ☐ B22 captions SPSS **H2 Purchase by Group**

Take-home · IN-CLASS_Lab Part C

- ☐ Review IN-CLASS_Lab Part A, IN-CLASS_Lab Part B and IN-CLASS_Lab Part C against the supplied formula explanations and visible diagnostic checks; retain your own written decision.
- ☐ B15 written: Decision; replace after class
- ☐ B19 written: H3 Viewer evidence caption

Part A · IN-CLASS_Lab Part A

- ☐ Cell B11 displays a p-value of 0.333.
- ☐ B13 statement chosen
- ☐ B14 written: Reason for your class choice
- ☐ B17 captions SPSS **H1 Satisfaction Benchmark**

Part C · IN-CLASS_Lab Part C

- ☐ Cell B13 displays -\$360 and B15 contains a provisional evidence-based decision sentence.

Whole workbook

- ☐ Every *Check your work* box reads "✓ Checked"
- ☐ No #NAME? , #VALUE! , #REF! , #NUM! or #DIV/0! anywhere
- ☐ Data sheets unchanged; formulas, not pasted values

Optional practice (not graded)

Everything here is optional and ungraded. Topic 1 is "Reproduce the in-class parts" and stops at the in-class calculations. Excel uses the workbook; SPSS uses the supplied .sps file. The Python and R scripts read the supplied CSV files in the current folder. They do not choose a dropdown answer or write a take-home decision. The later topics ask new questions. Work in a copy when changing a workbook. Start when you finish the practice.

Companion notebooks in your package:

- `S06P_Python_Exploratory.ipynb`
- `S06P_R_Exploratory.ipynb`

Topic 1 · Reproduce the in-class parts Reproduce

Same question in Excel → SPSS → Python → R.

Excel Trace Parts A, B, and C in the workbook Parts A, B, C

Reopen `IN-CLASS_Lab Part A`, `IN-CLASS_Lab Part B`, and `IN-CLASS_Lab Part C`. Check the formula cells and visible diagnostics. Trace the inputs for Part C B13 without changing your submitted workbook.

SPSS Compare H1, H2, and H3 with the workbook Parts A, B, C

Run the supplied `.sps` file and open its Viewer output. Match H1 to Part A, H2 to Part B, and H3 to Part C. For H2 and H3, read the uncorrected Pearson row when reconciling a two-proportion test.

Python Recompute the in-class tests and net impact Parts A, B, C

Run this in the folder with the supplied CSV files. It finds the survey and checkout columns, prints in-class calculations only, and makes no launch choice. It expects one row per eligible checkout user and one score per recorded survey response.

```
import glob, math, re
import pandas as pd

files = [pd.read_csv(path) for path in sorted(glob.glob('*.csv'))]
if not files:
    raise FileNotFoundError('Run this in the folder with the supplied CSV files.')
def key(s): return re.sub('[^a-z0-9]', '', str(s).lower())
def column(df, names):
    for c in df.columns:
        if key(c) in names: return c
    return None
score_names = {'satisfaction', 'satisfactionscore', 'satisfactionpoints', 'satscore'}
group_names = {'group', 'arm', 'variant', 'experimentgroup'}
buy_names = {'purchase', 'purchased', 'converted', 'conversion', 'orderplaced'}
complaint_names = {'complaint', 'complained', 'hascomplaint', 'complaintflag'}
surveys = [(d, column(d, score_names)) for d in files if column(d, score_names)]
checkouts = [(d, column(d, group_names), column(d, buy_names), column(d, complaint_names)) for d in files if
column(d, group_names) and column(d, buy_names) and column(d, complaint_names)]
if not surveys or not checkouts:
    raise ValueError('Check the supplied CSV headers for satisfaction, group, purchase, and complaint columns.')
survey, sc = surveys[0]
checkout, gc, bc, cc = checkouts[0]
scores = pd.to_numeric(survey[sc], errors='coerce').dropna()
n = len(scores); mean = scores.mean(); sd = scores.std(ddof=1)
if n < 2: raise ValueError('At least two recorded satisfaction scores are needed.')
t = (mean - 70) / (sd / math.sqrt(n))
df = n - 1
const = math.exp(math.lgamma((df+1)/2)-math.lgamma(df/2)) / math.sqrt(df*math.pi)
def density(v): return const * (1 + v*v/df)**(-(df+1)/2)
steps = 10000; h = abs(t)/steps
area = h/3 * (density(0) + density(abs(t)) + 4*sum(density(j*h) for j in range(1, steps, 2)) + 2*sum(density(j*h)
for j in range(2, steps, 2)))
p_t = max(0.0, 1 - 2*area)
print(f'Part A: n={n}, mean={mean:.3f}, SE={sd/math.sqrt(n):.3f}, t={t:.3f}, two-sided p={p_t:.4f}')
arm = checkout[gc].astype(str).str.strip().str.upper()
if not arm.isin(['A', 'B']).all(): raise ValueError('Checkout groups must be A or B.')
def yes(series):
    v = series.astype(str).str.strip().str.lower()
    if not v.isin(['1', '0', 'yes', 'no', 'y', 'n', 'true', 'false']).all():
        raise ValueError('Purchase and complaint flags must be recorded for each eligible user.')
    return v.isin(['1', 'yes', 'y', 'true'])
buy = yes(checkout[bc]); complaint = yes(checkout[cc])
na = (arm == 'A').sum(); nb = (arm == 'B').sum()
xa = int(buy[arm == 'A'].sum()); xb = int(buy[arm == 'B'].sum())
ca = int(complaint[arm == 'A'].sum()); cb = int(complaint[arm == 'B'].sum())
pa, pb = xa/na, xb/nb
pooled = (xa+xb)/(na+nb)
se0 = math.sqrt(pooled*(1-pooled)*(1/na+1/nb))
z = (pb-pa)/se0
p_z = math.erfc(abs(z)/math.sqrt(2))
```

```

se_effect = math.sqrt(pa*(1-pa)/na + pb*(1-pb)/nb)
print(f'Part B: A {xa}/{na}, B {xb}/{nb}; pooled={pooled:.4f}; z={z:.3f}; p={p_z:.4f}')
print(f'Part B: B-minus-A gap={pb-pa:.4f}; 95% interval=({pb-pa-1.96*se_effect:.4f}, {pb-pa+1.96*se_effect:.4f});
Pearson={z*z:.3f}')
ra, rb = ca/na, cb/nb
net = (xb-xa)*12 - nb*0.60 - (cb-ca)*25
print(f'Part C: complaints A {ca}/{na}, B {cb}/{nb}; gap={rb-ra:.4f}; net=${net:.2f}')

```

R Recompute the in-class tests and net impact Parts A, B, C

This base R script reads the supplied CSV files in the current folder. It prints the in-class quantities only. It does not fill a dropdown or decide whether to launch.

```

paths <- list.files(pattern = '\\.csv$', ignore.case = TRUE)
if (!length(paths)) stop('Run this in the folder with the supplied CSV files.')
files <- lapply(paths, function(p) read.csv(p, check.names = FALSE, stringsAsFactors = FALSE))
key <- function(x) gsub('^[a-z0-9]', '', tolower(x))
col <- function(d, names) { hit <- which(key(names(d)) %in% names); if (length(hit)) hit[1] else NA_integer_ }
sn <- c('satisfaction', 'satisfactionscore', 'satisfactionpoints', 'satscore')
gn <- c('group', 'arm', 'variant', 'experimentgroup')
bn <- c('purchase', 'purchased', 'converted', 'conversion', 'orderplaced')
cn <- c('complaint', 'complained', 'hascomplaint', 'complaintflag')
si <- which(vapply(files, function(d) !is.na(col(d, sn)), logical(1)))[1]
ci <- which(vapply(files, function(d) all(!is.na(c(col(d, gn), col(d, bn), col(d, cn)))), logical(1)))[1]
if (is.na(si) || is.na(ci)) stop('Check CSV headers for satisfaction, group, purchase, and complaint.')
survey <- files[[si]]; checkout <- files[[ci]]
scores <- suppressWarnings(as.numeric(survey[[col(survey, sn)]]))
scores <- scores[!is.na(scores)]; n <- length(scores)
if (n < 2) stop('At least two recorded scores are needed.')
se <- sd(scores)/sqrt(n); t <- (mean(scores)-70)/se
cat(sprintf('Part A: n=%d, mean=%.3f, SE=%.3f, t=%.3f, two-sided p=%.4f\n', n, mean(scores), se, t, 2*pt(-abs(t),
n-1)))
arm <- toupper(trimws(as.character(checkout[[col(checkout, gn)]])))
if (!all(arm %in% c('A', 'B'))) stop('Checkout groups must be A or B.')
yes <- function(v) {
  v <- tolower(trimws(as.character(v)))
  if (!all(v %in% c('1', '0', 'yes', 'no', 'y', 'n', 'true', 'false'))) stop('Purchase and complaint flags must be
recorded.')
  v %in% c('1', 'yes', 'y', 'true')
}
buy <- yes(checkout[[col(checkout, bn)]]); complaint <- yes(checkout[[col(checkout, cn)]])
na <- sum(arm=='A'); nb <- sum(arm=='B')
xa <- sum(buy[arm=='A']); xb <- sum(buy[arm=='B'])
ca <- sum(complaint[arm=='A']); cb <- sum(complaint[arm=='B'])
pa <- xa/na; pb <- xb/nb; pooled <- (xa+xb)/(na+nb)
se0 <- sqrt(pooled*(1-pooled)*(1/na+1/nb)); z <- (pb-pa)/se0
se.effect <- sqrt(pa*(1-pa)/na + pb*(1-pb)/nb)
cat(sprintf('Part B: A %d/%d, B %d/%d; pooled=%.4f; z=%.3f; p=%.4f\n', xa, na, xb, nb, pooled, z, 2*pnorm(-abs(z))))
cat(sprintf('Part B: gap=%.4f; 95% interval=(%.4f, %.4f); Pearson=%.3f\n', pb-pa, pb-pa-1.96*se.effect, pb-
pa+1.96*se.effect, z^2))
net <- (xb-xa)*12 - nb*0.60 - (cb-ca)*25
cat(sprintf('Part C: complaints A %d/%d, B %d/%d; gap=%.4f; net=$%.2f\n', ca, na, cb, nb, cb/nb-ca/na, net))

```

Topic 2 · How precise is the satisfaction mean? Extend

Same question in Excel → SPSS → Python → R.

Excel Add an interval for the recorded survey mean Parts A

In a copy of Part A, calculate a 95% interval in spare cells with `=B8-T.INV.2T(0.05,B12)*B9` and `=B8+T.INV.2T(0.05,B12)*B9`. Compare its bounds with the goal in D7. Explain why this interval still does not make the respondents representative.

SPSS Read the H1 mean interval Parts A

In the supplied `.sps` output, revisit H1's one-sample tables. Locate the confidence interval for the mean difference from the goal. Add the goal back to each bound if you want bounds for the mean itself.

Python Bootstrap the recorded mean Parts A

Resample the recorded survey scores to see how the sample mean moves. This is a precision exercise, not a cure for an unrepresentative survey.

```
import glob, re
import numpy as np
import pandas as pd

def key(x): return re.sub('[^a-z0-9]', '', str(x).lower())
names = {'satisfaction', 'satisfactionscore', 'satisfactionpoints', 'satscore'}
scores = None
for path in sorted(glob.glob('*.csv')):
    d = pd.read_csv(path)
    for c in d.columns:
        if key(c) in names:
            scores = pd.to_numeric(d[c], errors='coerce').dropna().to_numpy()
            break
    if scores is not None: break
if scores is None or len(scores) < 2: raise ValueError('Find the supplied CSV with recorded satisfaction scores.')
rng = np.random.default_rng(2026)
means = np.array([rng.choice(scores, size=len(scores), replace=True).mean() for _ in range(5000)])
print('Recorded mean:', scores.mean(), '95% bootstrap interval:', np.percentile(means, [2.5, 97.5]))
print('This interval does not establish that respondents represent all users.')
```

R Bootstrap the recorded mean Parts A

Use base R to resample the survey scores. Keep sampling precision separate from survey coverage.

```
key <- function(x) gsub('[^a-z0-9]', '', tolower(x))
namesWanted <- c('satisfaction', 'satisfactionscore', 'satisfactionpoints', 'satscore')
scores <- NULL
for (path in list.files(pattern='\\.csv$', ignore.case=TRUE)) {
  d <- read.csv(path, check.names=FALSE)
  hit <- which(key(names(d)) %in% namesWanted)
  if (length(hit)) { scores <- suppressWarnings(as.numeric(d[[hit[1]]])); scores <- scores[!is.na(scores)]; break
}
}
if (is.null(scores) || length(scores)<2) stop('Find the supplied CSV with recorded satisfaction scores.')
set.seed(2026)
means <- replicate(5000, mean(sample(scores, size=length(scores), replace=TRUE)))
print(c(recorded.mean=mean(scores), quantile(means, c(.025, .975))))
cat('This interval does not establish that respondents represent all users.\n')
```

Topic 3 · Are purchases and complaints associated within each arm? Extend

Same question in Excel → SPSS → Python → R.

Excel Make two purchase-by-complaint tables Parts B, C

In a copy, make a PivotTable from `Checkout`. Filter to A and place purchase and complaint flags on rows and columns; repeat for B. Show counts, not just percentages. For each two-by-two table, compare observed counts with counts expected from its row and column totals before using `CHISQ.TEST`.

SPSS Separate the two arm tables Parts B, C

After running the supplied `.sps`, use the checkout data it opens. Split output by group, then run `CROSSTABS` with purchase by complaint, observed and expected counts, and Pearson chi-square. Turn Split File off afterward. An association within an arm does not establish its cause.

Python **Inspect within-arm two-by-two tables** Parts B, C

This looks for a purchase–complaint association separately in A and B. It is not another estimate of the checkout treatment effect.

```
import glob, math, re
import pandas as pd

def key(x): return re.sub('[^a-z0-9]', '', str(x).lower())
def find(d, options): return next((c for c in d.columns if key(c) in options), None)
group = {'group','arm','variant','experimentgroup'}
buy = {'purchase','purchased','converted','conversion','orderplaced'}
complaint = {'complaint','complained','hascomplaint','complaintflag'}
found = None
for path in sorted(glob.glob('*.csv')):
    d = pd.read_csv(path)
    cols = [find(d, group),find(d, buy),find(d, complaint)]
    if all(cols): found = (d, cols); break
if found is None: raise ValueError('Find group, purchase, and complaint columns in the supplied CSV files.')
d, (gc,bc,cc) = found
for arm in ('A','B'):
    part = d[d[gc].astype(str).str.strip().str.upper().eq(arm)]
    table = pd.crosstab(part[bc].astype(str).str.lower(), part[cc].astype(str).str.lower())
    print('\nArm',arm,'\n',table)
    if table.shape != (2,2):
        print('A two-by-two Pearson test needs both observed flag values.');
```

continue

```
    obs = table.to_numpy(); expected = obs.sum(axis=1)[:,None]*obs.sum(axis=0)[None,:]/obs.sum()
    if (expected == 0).any():
        print('An expected cell is zero; do not calculate Pearson chi-square.');
```

continue

```
    chi2 = ((obs-expected)**2/expected).sum()
    print('Pearson chi-square:',chi2,'p:',math.erfc(math.sqrt(chi2/2)))
print('Association within an arm does not establish cause.')
```

R Inspect within-arm two-by-two tables Parts B, C

Use a separate purchase-by-complaint table for each arm. Treat small expected counts as a reason to question the Pearson approximation.

```
key <- function(x) gsub('[^a-z0-9]', '', tolower(x))
find <- function(d, choices) { hit <- which(key(names(d)) %in% choices); if(length(hit)) hit[1] else NA_integer_
}
gn <- c('group','arm','variant','experimentgroup')
bn <- c('purchase','purchased','converted','conversion','orderplaced')
cn <- c('complaint','complained','hascomplaint','complaintflag')
d <- NULL
for (path in list.files(pattern='\\.csv$', ignore.case=TRUE)) {
  candidate <- read.csv(path, check.names=FALSE, stringsAsFactors=FALSE)
  if(all(!is.na(c(find(candidate,gn),find(candidate,bn),find(candidate,cn)))) { d <- candidate; break }
}
if(is.null(d)) stop('Find group, purchase, and complaint columns in the supplied CSV files.')
arm <- toupper(trimws(as.character(d[[find(d,gn)]])))
for (a in c('A','B')) {
  rows <- arm==a
  tab <- table(Purchase=d[[find(d,bn)]] [rows], Complaint=d[[find(d,cn)]] [rows])
  cat('\nArm',a,'\n'); print(tab)
  if(all(dim(tab)==c(2,2)) && all(rowSums(tab)>0) && all(colSums(tab)>0)) {
    test <- suppressWarnings(chisq.test(tab, correct=FALSE))
    print(test$expected); print(test)
  } else cat('A two-by-two Pearson test needs both observed flag values.\n')
}
cat('Association within an arm does not establish cause.\n')
```

Topic 4 • Was the intended assignment balance preserved? Extend

Same question in Excel → SPSS → Python → R.

Excel Audit a planned equal split Parts B

If the assignment plan specified equal arms, use Part B B7:B8 to find the expected count per arm: $(B7+B8)/2$. In a copy, calculate the sum of each arm's squared observed-minus-expected gap divided by that expected count. Apply `CHISQ.DIST.RT` with one degree of freedom. This checks allocation balance, not purchase performance.

SPSS Check counts against the assignment plan Parts B

Use the checkout dataset opened by the supplied `.sps`. First inspect group frequencies for missing or duplicate user records. If the documented plan was an equal split, compare the observed A and B counts with that plan using a one-sample chi-square goodness-of-fit test. Do not infer the plan from the observed counts.

Python Check a documented equal-allocation plan Parts B

Use this only if the assignment plan called for equal A and B arms. This balance audit says nothing by itself about the purchase effect.

```
import glob, math, re
import pandas as pd

def key(x): return re.sub('[^a-z0-9]', '', str(x).lower())
d = None; gc = None
for path in sorted(glob.glob('*.csv')):
    candidate = pd.read_csv(path)
    names = {key(c):c for c in candidate.columns}
    hit = next((names[x] for x in ('group','arm','variant','experimentgroup') if x in names), None)
    if hit is not None: d, gc = candidate, hit; break
if d is None: raise ValueError('Find the supplied checkout CSV with a group column.')
arm = d[gc].astype(str).str.strip().str.upper()
if not arm.isin(['A','B']).all(): raise ValueError('Inspect missing or unexpected assignment labels first.')
a, b = int(arm.eq('A').sum()), int(arm.eq('B').sum())
expected = (a+b)/2
chi2 = (a-expected)**2/expected + (b-expected)**2/expected
print('Eligible assignment counts:',a,b)
print('Equal-split chi-square:',chi2,'p:',math.erfc(math.sqrt(chi2/2)))
print('Use this check only if equal allocation was the documented plan.')
```

R Check a documented equal-allocation plan Parts B

Compare eligible-user arm counts with an equal split only if that was the assignment plan. This is an assignment audit, not an outcome test.

```
key <- function(x) gsub('[^a-z0-9]', '', tolower(x))
d <- NULL; gc <- NULL
for(path in list.files(pattern='\\.csv$', ignore.case=TRUE)) {
    candidate <- read.csv(path, check.names=FALSE, stringsAsFactors=FALSE)
    hit <- which(key(names(candidate)) %in% c('group','arm','variant','experimentgroup'))
    if(length(hit)) { d <- candidate; gc <- hit[1]; break }
}
if(is.null(d)) stop('Find the supplied checkout CSV with a group column.')
arm <- toupper(trimws(as.character(d[[gc]])))
if(!all(arm %in% c('A','B'))) stop('Inspect missing or unexpected assignment labels first.')
counts <- table(factor(arm, levels=c('A','B')))
print(counts)
print(chisq.test(counts, p=c(.5,.5)))
cat('Use this check only if equal allocation was the documented plan.\n')
```